A Scalable Trie Building Algorithm for High-Throughput Phyloanalysis of Wafer-Scale Digital Evolution Experiments

Vivaan Singhvi[1,5,6,†], Joey Wagner[6,9], Emily Dolson[7,8,9], Luis Zaman[2,3,5], and Matthew Andres Moreno[2,3,4,5,‡]

[1]Michigan Research and Discovery Scholars [2]Department of Ecology and Evolutionary Biology
[3]Center for the Study of Complex Systems [4]MIDAS [5]University of Michigan, Ann Arbor, United States
[6]Professorial Assistantship Program [7]Department of Computer Science and Engineering
[8]Program in Ecology, Evolution, and Behavior [9]Michigan State University, East Lansing, United States
[†]vsinghvi@umich.edu [‡]morenoma@umich.edu

Abstract

Agent-based simulation platforms play a key role in enabling fast-to-run evolution experiments that can be precisely controlled and observed in detail. Availability of high-resolution snapshots of lineage ancestries from digital experiments, in particular, is key to investigations of evolvability and open-ended evolution, as well as in providing a validation testbed for bioinformatics method development. Ongoing advances in AI/ML hardware accelerator devices, such as the 850,000-processor Cerebras Wafer-Scale Engine (WSE), are poised to broaden the scope of evolutionary questions that can be investigated *in silico*. However, constraints in memory capacity and locality characteristic of these systems introduce difficulties in exhaustively tracking phylogenies at runtime. To overcome these challenges, recent work on hereditary stratigraphy algorithms has developed space-efficient genetic markers to facilitate fully decentralized estimation of relatedness among digital organisms. However, in existing work, compute time to reconstruct phylogenies from these genetic markers has proven a limiting factor in achieving large-scale phyloanalyses. Here, we detail an improved trie-building algorithm designed to produce reconstructions equivalent to existing approaches. For modestly-sized 10,000-tip trees, the proposed approach achieves a 300-fold speedup versus existing state-of-the-art. Finally, using 1 billion genome datasets drawn from WSE simulations encompassing 954 trillion replication events, we report a pair of large-scale phylogeny reconstruction trials, achieving end-to-end reconstruction times of 2.6 and 2.9 hours. In substantially improving reconstruction scaling and throughput, presented work establishes a key foundation to enable powerful high-throughput phyloanalysis techniques in large-scale digital evolution experiments.

Introduction

Key aspects of the study of evolution, whether biological or digital, revolve around understanding the flow of genetic material among large populations of organisms. As such, phylogenetic analyses assessing ancestry trees representing organisms' evolutionary histories are a core tool in evolutionary biology.

In biology, phylogeny estimations are typically reconstructed through *post hoc* analysis of genetic similarities among organisms. In contrast, direct, exact tracking at runtime is typical in *in silico* experiments. However, in memory-constrained parallel and distributed computing contexts, *post hoc* reconstruction approaches can become advantageous owing to runtime synchronization and storage costs of direct tracking.

Akin to biological studies, efficacy of phylogenetic analysis in such digital experiments hinges on fast, accurate methods to estimate ancestry trees from genome data. In this work, we present a novel trie-building algorithm that greatly reduces compute time necessary to reconstruct phylogenies from special-purpose markers on digital genomes, while producing results equivalent to a naive approach.

Applications of Phylogenetic Analysis

Phylogenetic analyses provide key means to characterize and quantify a broad array of evolutionary processes. Classically, these analyses have been applied to investigation of species-level macroevolutionary dynamics revolving around speciation and extinction rates; however, population- and organism-level dynamics can also be inferred, such as the spread of beneficial mutations within a population or fitness parameters like growth rate and probability of survival (Genthon et al., 2023; Levy et al., 2015; Stadler, 2013). Phylogenetic analysis is also crucial in the field of epidemiology, playing a key role in informing public health interventions. In this context, phylogenetic methods can be used to determine transmission history, pinpointing where and how chains of infection unfold (Wang et al., 2020). In this vein, phylogenies are also key in assessing the prevalence of "super-spreader" dynamics wherein disease spread is driven by a small set of high-risk individuals (Colijn & Gardy, 2014).

Phylogenies and Digital Evolution

In some cases, aspects of biological evolution can be difficult or infeasible to observe on human timescales; laboratory experiments may take years, or even decades, to complete (Wiser et al., 2013; Stroud & Ratcliff, 2025). By simulating the behavior of a population, some experiments can instead be conducted digitally — often completing in a fraction of the time. Digital experiments can model key characteristics of biological populations, such as variation, natural selection, ecological interactions, spatial distribution, and more (Dolson & Ofria, 2021; Haller & Messer, 2023). As such, conclusions from digital evolution experiments can contribute meaningfully to understanding biology (Pennock, 2007).

In digital evolution contexts, phylogenies have likewise proven valuable. In application-oriented contexts, phylogeny-based biodiversity metrics have been shown as predictive of

Downloaded from direct.mit.edu/isal/proceedings-pdf/isal2025/37/62/2567097/isal_a_890.pdf by guest on 14 October 2026

solution quality outcomes for evolutionary computation-based optimization (Hernandez et al., 2022). Phylogeny-based methods can also be applied to characterize more general aspects of ecology, spatial structure, and selection pressure within *in silico* populations (Moreno et al., 2023).

Digital evolution approaches can also serve as a testbed to assess bioinformatics methodologies. The Aevol_4b system, for instance, uses a genetic system corresponding to that of DNA, allowing any genetic information to be processed using methods directly from bioinformatics (Daudey et al., 2024). Likewise, population genetics work often incorporates SLiM, which supports sophisticated continuous-space modeling of single- and multi-species systems (Haller & Messer, 2023).

Given the programmatic observability of digital simulations, digital evolution platforms typically incorporate direct tracking methods that record lineage ancestry as the simulation runs. General-purpose phylogeny-tracking libraries exist for this purpose (Dolson et al., 2024), although many platforms simply incorporate bespoke implementations into their own software (Ofria & Wilke, 2004).

Scaling Up Digital Evolution Experiments

To achieve large-scale digital evolution experiments, it is necessary to move from a single-processor system to a more distributed approach with many computing units (Moreno et al., 2024b). In large-scale, many-processor simulations, however, challenges arise in managing a comprehensive record of ancestry. To control memory use, it is typically necessary to trim away records of extinct lineages when performing direct tracking. Detecting extinctions, however, introduces implementation complexity and overhead costs when lineage histories span across multiple processors. Exhaustive tracking is also sensitive to data loss from crashed hardware or dropped messages, which has been highlighted as a key consideration in achieving very large-scale artificial life systems (Ackley, 2016; Ackley & Small, 2014).

Challenges associated with comprehensive tracking are especially acute in specialized hardware accelerator devices, which represent a promising emerging direction in high-performance computing (Emani et al., 2024). In incorporating thousands of processor cores per device, these hardware architectures impose trade-offs in memory capacity limitations and data locality restrictions that limit the feasibility of comprehensive tracking. In such contexts, reconstruction-based approaches can provide an attractive balance between data fidelity and data collection overhead.

Hereditary Stratigraphy

Under controlled conditions, such as laboratory experiments or evolution simulations, genetic material may be engineered to facilitate the accuracy and efficiency of estimating phylogenetic relatedness (Li et al., 2024; Ackley, 2023).[1] Work developing hereditary stratigraphy ("hstrat") methods seeks to operate analogously, providing techniques to organize genetic material in digital organisms that maximize reconstruction quality while minimizing memory footprint (Moreno et al., 2022a). Hereditary stratigraphy components can be bundled with agent genomes in a manner akin to non-coding DNA (i.e., neutral with respect to agent traits and fitness), enabling generalizability across a wide variety of agent models.

Hereditary stratigraphy associates each generation along each lineage with an identifying "fingerprint" marker, referred to as a differentia. On birth, each offspring receives a new differentia value and appends it to an inherited chronological record of past values — corresponding to earlier generations along its lineage. Under this scheme, mismatching differentia can be used to delimit the end of common ancestry between two organisms. Figure 1 summarizes this approach.

To save space, differentiae may be pruned away — although, at the cost of reducing precision in inferring relatedness. Using fewer bits per differentia can also provide many-fold memory savings; single bits or single bytes are appropriate for most use cases.

While inferring relatedness from biological sequence data can be a highly challenging and computationally-intensive problem (Miller et al., 2010), the structured marker data used in hereditary stratigraphy somewhat ameliorates this challenge by allowing phylogeny reconstruction to be approached as a trie-building problem of identifying common string prefixes (De La Briandais, 1959; Moreno et al., 2024a). However, the presence of missing data due to some differentia being dropped to save memory complicates matters.

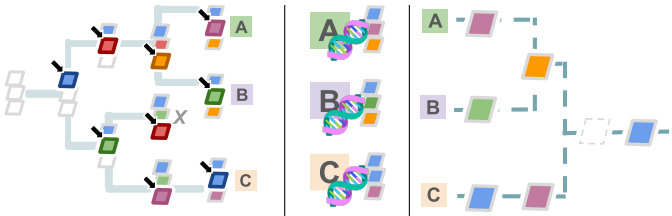
In the context of trie-building, missing marker time points (possessed by only a subset of organisms) effectively act as "wildcard" characters in prefix matching operations. Therefore, placing an organism on a trie requires evaluating diverging string paths beyond the wildcard to identify further matches. Given the likelihood of differentia value collisions for small differentia sizes (e.g., 1 bit), identifying the best-matching path after a wildcard value can require looking ahead several consecutive markers. Furthermore, where consecutive wildcard values are encountered, the number of possible paths that must be explored can grow exponentially.

Although previous work has investigated the quality of phylogenies constructed from hereditary stratigraphy data using trie-based approaches (Moreno et al., 2025b), the computational intensity of the naive wildcard-matching approach has limited the scale of phylogenetic reconstructions investigated and restricted experimental throughput for smaller reconstructions. Given the objective of hereditary stratigraphy methodology to facilitate studying very large-scale digital evolution experiments, achieving reconstruction efficiency sufficient for large-scale phyloanalysis is critical to the overall utility of the methodology in enabling observable experiments.

In this work, we describe an algorithm for efficient trie reconstruction in the face of missing data, and explore its performance characteristics. The following section introduces our proposed "shortcut" algorithm for the trie building approach explored in this paper. We then detail methods and results for benchmark trials assessing empirical scaling behavior and performance on large-scale billion-genome workloads.

[1] Notably, Ackley has applied barcoding approaches to track recent ancestry among emergent replicators in a distributed fabric-computing context.

Downloaded from direct.mit.edu/isal/proceedings-pdf/isal2025/37/62/2567097/isal_a_890.pdf by guest on 14 October 2026



(a) evolve (b) sample (c) reconstruct

Figure 1: **Overview of hereditary stratigraphy.** At runtime, genomes are annotated with randomly-generated heritable markers (panel 1a). To maintain fixed-memory footprint, some markers are overwritten. Genomes of interest are sampled at runtime and from end state (panel 1b). Decoded genome markers enable estimation of evolutionary relatedness (panel 1c), subject to error from marker-value collisions and discarded markers.

Proposed Algorithm

In this section, we overview the existing naive approach used for constructing prefix tries of hereditary stratigraphy markers, and describe the proposed shortcut-based approach developed in this work. Notation describes hereditary stratigraphy markers as a pair of two properties: *rank* r, referring to the generation at which a marker was generated, and *differentia* d, referring to the randomly-generated value distinguishing a given marker from others generated at that generation.

Naive Trie-Building Algorithm

The naive algorithm treats each organism's sequence of stored rank-differentia pairs as a string: each differentia d constitutes a character at the rank r-th string position. Differentia values missing due to having been overwritten are considered as wildcard characters.

For two rank-differentia strings, common ancestry corresponds to a shared prefix of string characters inherited from their most recent common ancestor (MRCA). Under this framing, phylogenies correspond to a trie — a tree-like data structure where each node corresponds to a character in a string, and each path from the root to a leaf node reads out a stored string (Fredkin, 1960).

The first step in naive reconstruction is to sort all organisms in ascending order by number of generations elapsed. Due to each organism using the same algorithm to determine which markers to overwrite, this order guarantees that any data missing from an organism will also have been discarded by all subsequent organisms.

We initialize an empty trie, consisting of a single root node. Trie building proceeds by processing organisms one at a time in their sorted order. For each organism, we iterate through rank-differentia pairs in chronological order. At each rank-differentia pair, we either continue along the existing trie, branch off to create a new path, or — in a special case — address missing information (Algorithm 1).

Missing information is recognized as follows. Suppose we have reached trie node n with rank r_1 and must next process rank-differentia pair (r_2, d) from current organism o. If any node with rank $r' < r_2$ exists among children of current node n, we know that r' must have been deleted from organism o as its record skips directly from r_1 to r_2.

To maximize reconstruction accuracy in the case of missing data, we must assess which — if any — of n's children our missing datum at r' would have likely corresponded to. Essentially, when reaching missing

```
1:  function RECONSTRUCTTREE(organism list O)
2:      T ← an empty tree
3:      for o∈O do
4:          TREEINSERT(T, o)
5:      return T
6:  function TREEINSERT(tree T, organism o)
7:      n ← root of T
8:      for (r,d)∈o do   ▷ successive rank-differentia pairs
9:          n ← MOSTLIKELYCHILD(n,o)   ▷ see supplement
10:         if ∃c∈children(n) s.t. differentia(c)=d and rank(c)=r
            then
11:             n ← c
12:         else
13:             c′ ← CREATECHILD(n,r,d)   ▷ new child off n
14:             n ← c′
15:     o.parent ← n   ▷ attach organism o to tree
```

Algorithm 1: **Naive trie-building algorithm.** Iteratively builds a trie from organisms' genetic markers. Requires a list of organisms O in ascending order by generations elapsed. Within organisms, genetic markers are stored as a chronological list of rank-differentia (r,d) pairs.

information, we must infer that information, which we do by looking ahead in the tree. Specifically, we search forward for the path with the longest successive streak of differentia matching to current organism o (Moreno et al., 2024a). If there is no matching path, we simply branch off node n rather than continuing to traverse through its children.

Note that this approach suffers the cost of doing significant extra work to handle missing information. In the worst case, there could be an exponential number of matching paths to check, owing to the possibility of nested branch-outs at successive wildcard sites. These searches are repeated for each organism, regardless of whether or not a similar search was already done. Therefore, we present an improved algorithm that consolidates this extra work into a single step that never needs to be repeated.

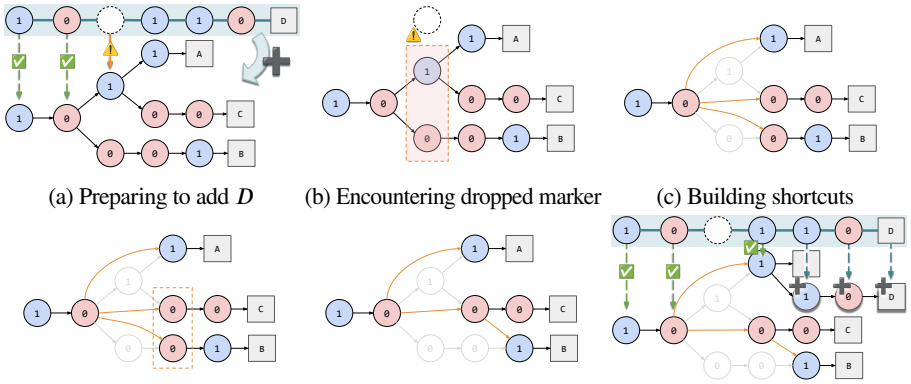
Proposed Shortcut Algorithm

We propose a modification of Algorithm 1 that replaces the MOSTLIKELYCHILD search in line 9 with a shortcut-building and consolidation step to deal with missing ranks.

Recall that once data from a specific rank is absent, it will not appear in any subsequent organisms. Hence, trie nodes corresponding to that rank are no longer informative in placing subsequent data. By building shortcuts bypassing such irrelevant nodes, we can speed up trie traversal for subsequent inserts.

In the shortcut-enabled approach, we detect missing information the same way as before: encountering some trie

Downloaded from direct.mit.edu/isal/proceedings-pdf/isal2025/37/62/2567097/isal_a_890.pdf by guest on 14 October 2026



(a) Preparing to add D (b) Encountering dropped marker (c) Building shortcuts

(d) Indistinguishable nodes (e) Collapsing redundant shortcuts (f) Organism D added

Figure 2: **Trie-consolidation procedure for proposed shortcut algorithm.** A dropped hereditary marker is encountered while extending trie with genome D (panel 2b). All subsequent-added genomes will also have dropped markers at this position, so corresponding trie nodes may be bypassed by "shortcut" connections (panel 2c). Note that bypassed trie structure is retained ("grayed-out" nodes), so corresponding phylogenetic structure remains when reconstruction is finalized. In a final step, shortcuts leading to indistinguishable nodes are further consolidated (panels 2d and 2e).

node n' with a rank r' that falls between the current organism o's last-inserted rank r_1 and its currently-inserting rank r_2 (i.e., $r_1 < r' < r_2$).

Suppose we detect missing information while stepping from trie node n. To proceed with shortcut construction, we will flesh out a full set B of nearby no-longer-informative nodes. We build B by collecting all descendants of n with rank r'' falling in the gap $r_1 < r'' < r_2$. We then create shortcut edges that bypass nodes in B — connecting n directly to the immediate descendants of B with rank $\geq r_2$.

For illustration, see Figure 2, where we detect a dropped rank (panel 2b) and create shortcut edges trie nodes with that dropped rank (panel 2c). Edges to bypassed nodes B are ignored in subsequent insert traversals, in favor of added shortcut edges.

One complicating factor may arise in shortcut construction: the introduction of "indistinguishable nodes," where new sibling nodes happen to share the same rank and differentia value (like panel 2d). This situation can be resolved by arbitrarily choosing one node to discard and transferring its children to the kept node using shortcut edges — essentially 'merging' the duplicates (panel 2e).

Algorithm 2 outlines steps for the described shortcut construction and consolidation procedure.

After insertion of the final organism into our trie, all that remains is to export a final phylogeny output. At this stage, we ignore shortcut edges. Crucially, while shortcut edges are useful for efficient traversal during insertions, they do not represent meaningful lineage relationships. Instead, we use original trie edges as the basis for phylogeny output — thereby preserving the full detail and granularity of the naive approach. It is therefore necessary to retain records of the original trie structure, represented by light gray nodes and edges in Figure 2. Details about algorithm implementation and more detailed discussion of naive equivalency are provided in supplemental material (Singhvi et al., 2025).

```
1:  function ConsolidateTree(tree T, node n, rank r)
2:      S ← {c ∈ descendants(n)
                : rank(c) ≥ r and rank(parent(c)) < r}
3:      for c in S do BuildShortcut(T, n, c)
4:      for (r, d) ∈ {(rank(s), differentia(s)) : s ∈ S} do
5:          S' ← {s ∈ S : rank(s) = r and differentia(s) = d}
6:          if |S'| = 1 then
7:              continue
8:          c* ← an arbitrary element in S'
9:          for c' ∈ S' \ {c*} do
10:             for c'' in children(c') do
11:                 BuildShortcut(T, c*, c'')
12:             RemoveShortcut(T, n, c')
13: function BuildShortcut(tree T, node n, node c)
14:     edges(T) ← edges(T) ∪ {(n, c)}
15:     edges(T)[(n, c)].is_shortcut ← True
16: function RemoveShortcut(tree T, node n, node c)
17:     edges(T) ← edges(T) \ {(n, c)}
```

Algorithm 2: **Shortcut-building and consolidation procedure.** Builds shortcuts from a given node n and collapses "indistinguishable" sibling sets introduced by those shortcuts.

Methods

In this section, we detail the methodology for benchmark and validation experiments used to assess the performance of the proposed shortcut-enabled algorithm.

Generation of Benchmark Data

Across our benchmark experiments, we needed a large-scale sample of genomes to perform reconstructions on. For this purpose, we performed simulations using the Cerebras CS-2 Wafer-Scale Engine, an 850,000-core hardware accelerator device (Lie, 2023). Processor elements (PEs) in this architecture are arranged in a grid lattice, with each processor able to execute independently and communicate with neighboring PEs through message-passing.

To conduct these simulations, we used an extension of the island-model genetic algorithm framework presented in (Moreno et al., 2024b). This framework organizes simulated organisms into independent subpopulations hosted on each PE. Synchronous tournament selection of size 2 is applied within each PE. Between generations, agents are migrated

Downloaded from direct.mit.edu/isal/proceedings-pdf/isal2025/37/62/2567097/isal_a_890.pdf by guest on 14 October 2026

between PEs asynchronously.

A simple genome model was used, comprised of a single floating point value representing fitness. Reproduction occurred asexually, with this value subject to a normally-distributed mutation with 30% probability. (In the case of the purifying-regime treatment, the negative absolute value of a sampled mutation delta was applied.) In tournament competitions to select parents for the subsequent generation, the parent was selected according to the higher fitness value with 10% probability and was otherwise selected randomly. Genomes were augmented with hstrat annotations comprising 64 single-bit markers, managed at runtime using a `tilted` curation policy, which favors dense retention of more recent marker data (Moreno et al., 2024c).

To generate data representative of differing evolutionary conditions, two treatments were applied: an *adaptive regime* and a *purifying regime*. Under the adaptive regime, mutations increasing fitness were allowed, introducing the possibility of selective sweeps. By contrast, the purifying regime treatment allowed only deleterious mutations. Previous work with this system demonstrated the purifying regime to exhibit substantially greater phylogenetic richness, in line with expectations for selective sweeps to diminish population-level phylogenetic diversity (Moreno et al., 2024b).

The simulation was run for 5 million generation cycles. A per-PE population size of 256 organisms was used, providing a net population size of 190.8 million organisms. "Fossil" genome data was sampled on a rolling basis over the course of simulation using asynchronous `memcopy` operations, where sample genomes were copied from each PE on a rolling basis. In total, collection of 1,999 (purifying regime) and 2,014 (adaptive regime) `memcopy` cycles elapsed. Real-time runtime under each treatment was 24 minutes.

For implementation-level details on the Wafer-Scale Engine simulation, see (Moreno et al., 2024b).

Microbenchmark Experiments

In a first set of microbenchmark trials, we performed a head-to-head comparison of runtime under the proposed shortcut-enabled algorithm versus the previous naive approach. Given performance limitations of the naive approach, the maximum reconstruction workload tested was limited to 10,000 genomes. Experiments used random subsamples of genomes from WSE fossil data described above. To facilitate informative comparison, timings measured just the trie-building component of reconstructions, excluding pre- and post-processing steps. Five replicate observations were taken per timing.

In a second set of microbenchmark trials, we conducted an empirical scaling analysis of the shortcut-enabled trie building algorithm across a wider spread of reconstruction workload sizes. To assess the relationship between marginal throughput and trie size, we performed trie reconstructions of 10 million WSE genomes, broken into successive batches of 100,000 genomes. This approach allowed us to independently measure time taken to insert each batch, to determine how this time changes for later batches (i.e., larger trees). We also performed unbatched tests evaluating net execution time to build tries ranging from 1 to 10 million tips. For these experiments, 3 replicates were performed per observation.

All microbenchmarks were replicated using genomes generated under adaptive and purifying regimes to assess the sensitivity of performance to underlying phylogeny structure.

Macrobenchmark Experiments

To assess the performance of our approach on large-scale workloads under real-world conditions, we supplemented microbenchmarks focusing on core trie extension and consolidation operations with two macrobenchmark trials comprising our full end-to-end reconstruction implementation (detailed below). To reduce peak memory use, we additionally included a step to collapse unnecessary unifurcations (i.e., parent nodes with only a single child) between trie-building batches — for more details, see the supplemental (Singhvi et al., 2025). These end-to-end experiments used the `hstrat.dataframe.surface_build_tree` CLI module incorporated in the `hstrat` library.

These experiments were conducted using the full billion-genome corpus of fossil data gathered from Wafer-Scale Engine simulations. We performed two trial reconstructions: one using genome data from the adaptive regime treatment and another under the purifying regime treatment. Reconstructions were conducted on 100-core cluster node allocations with 2 TB of memory.

Validation Tests

By design, our proposed shortcut-enabled algorithm produces equivalent reconstruction results compared to the reference naive approach, up to arbitrary tie-breaking decisions (see supplemental material (Singhvi et al., 2025) for detailed discussion). To confirm correctness of our implementation, however, we report several validation tests.[2]

Validation tests employed small evolutionary simulations run locally, to allow collection of ground-truth data via `phylotrackpy` (Dolson et al., 2024). Simulations propagated populations of 100 agents under neutral evolution conditions, with runs lasting 500 generations. We configured these simulations to consider both synchronous and asynchronous generations. Reconstructions over end-state genomes were tested, as well as reconstructions incorporating fossil genomes sampled at 50-generation intervals. To assess reconstruction quality, we calculated error as triplet distance between the ground truth and reconstructed phylogenies (Critchlow et al., 1996).

End-to-end Reconstruction Implementation

Figure 3 overviews the full end-to-end pipeline used to create a phylogenetic reconstruction from raw hstrat-annotated genome data. First, hstrat annotation data is extracted and decoded. To take advantage of SIMD parallelism, the decoding of origin times for genome markers is performed using bulk operations orchestrated via NumPy, batched over available

[2]In addition to validation experiments reported here, the `hstrat` library includes an extensive unit tests that compare reconstructed most recent common ancestor (MRCA) between organisms to the value expected by direct comparison of their hstrat marker records.

Downloaded from direct.mit.edu/isal/proceedings-pdf/isal2025/37/62/2567097/isal_a_890.pdf by guest on 14 October 2026

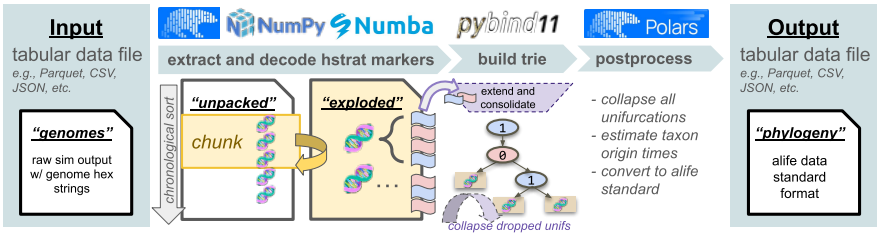


Figure 3: **Phylogeny pipeline.** Hstrat markers are "unpacked" from genome hex strings. Individual markers are decoded into "exploded" format. Inferred trie are progressively extended with genomes, purging dropped unifurcations. Finally, trie is exported to ALife data standard. High-performance scientific computing libraries are leveraged to support large datasets.

processors using the Numba library threading engine. Due to memory intensity of the exploded representation, where each genome marker constitutes an individual dataframe row, a chunk size may be configured to limit the amount of data exploded at any one time. A chunk size of 50 million genomes was used for macrobenchmark trials.

Subsequently, decoded marker data is fed genome-by-genome into the trie-building backend. To ensure full hardware utilization, execution of the core trie-building procedure — which is single-threaded — is performed concurrently with decode/explode work on the upcoming chunk. Finally, a phylogeny is extracted from the end-state trie, converted to ALife standard format, and saved to disk. The Polars library is used for load/save and other pre-/post-processing operations, allowing some operations to be distributed over available processor cores by the underlying threading engine.

Software, Data, and Materials

Software used in this work is available via GitHub and archived via Zenodo.[3] Data and supplementary material are hosted via the Open Science Framework at `osf.io/63ucz` (Singhvi et al., 2025; Foster & Deardorff, 2017). All accompanying materials are provided open-source under the MIT License.

Microbenchmark experiments were conducted on a 2019 MacBook Pro (2.3GHz 8-Core Intel i9, 16GB RAM) using a benchmark script provided in supplemental material (Singhvi et al., 2025). Head-to-head trie-building comparisons were performed using Python implementations of both algorithms. Empirical scaling analysis experiments used a C++ implementation of the shortcut-enabled trie-building algorithm.

Macrobenchmarks were performed on AMD EPYC compute cluster nodes with 128 cores and 2005 GB of memory. Purifying- and adaptive-regime reconstructions were performed on separate nodes with EPYC 7H12 (2.595 GHz) and EPYC 7763 (2.445 GHz) processors, respectively. End-to-end experiments also used a C++ implementation of the shortcut-enabled trie-building algorithm.

This project benefited significantly from open-source scientific software (Virtanen et al., 2020; Harris et al., 2020; pandas developers, 2020; McKinney, 2010; Waskom, 2021; Hunter, 2007; Moreno et al., 2022b; Yang et al., 2025).

[3] Benchmarking and validation code (v1.0.1) is available on GitHub at `mmore500/hstrat-reconstruction-algo` (Moreno et al., 2025c), WSE kernel code (v2025.12.26) for Cerebras SDK v1.0.0 at `mmore500/wse-async-ga` (Moreno et al., 2025d), and hstrat library code (v1.20.10) at `mmore500/hstrat` (Moreno et al., 2025a).

Results and Discussion

Microbenchmark: Runtime Comparison

Direct comparison of runtimes across a range of problem sizes demonstrated substantial improvements under shortcut-enabled trie building, compared to the naive approach. Figure 4 compares execution time for both approaches for problem sizes ranging up to 10,000 genomes. Over this window, the naive algorithm's runtime appears to grow at a superlinear pace. At the largest surveyed problem size, naive runtime reaches 15 minutes. In contrast, runtime for the shortcut-enabled approach at this scale is approximately 3 seconds — a 300-fold difference in performance.

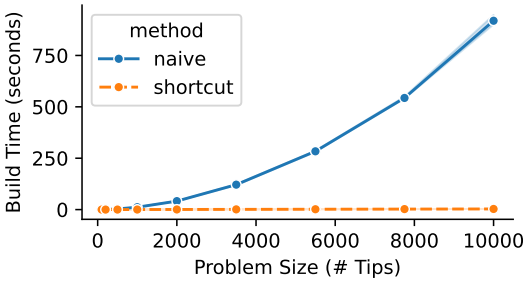
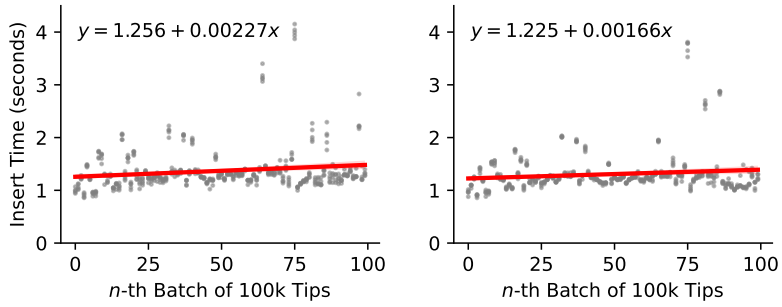


Figure 4: **Microbenchmark comparison of naive trie and shortcut table algorithms.** Naive approach appears to exhibit superlinear scaling. Shaded bands are 100% percentile intervals, with 5 samples per observation.

Although the same programming language was used for both implementations in this trial to ensure apples-to-apples comparability, implementations did differ in the underlying data structure used to store trie data. The shortcut-enabled implementation used a contiguous edge-list table to store trie data, while the naive implementation used a node-and-pointer approach. Typically, table-based approaches are faster in practice due to cache locality effects and reduced memory allocation calls. However, both data structures provide equivalent time complexities for all relevant initialization and lookup operations. Thus, the notable difference in scaling properties between the shortcut-enabled and naive benchmarks is indicative of meaningful differences in the underlying algorithms, beyond effects attributable to data structure implementations.

Downloaded from direct.mit.edu/isal/proceedings-pdf/isal2025/37/62/2567097/isal.a.890.pdf by guest on 14 October 2026



(a) Purifying Regime (b) Adaptive Regime

Figure 5: **Empirical performance scaling of shortcut table algorithm in microbenchmark trials.** Panels 5a and 5b differ in simulation data source of sampled genomes, with the former exhibiting higher phylogenetic richness. Data collected as batches are added in a 10 million tip reconstruction, sampled 5 times per panel.

Microbenchmark: Empirical Scaling Analysis

Having seen a substantial speedup in comparing the shortcut-enabled algorithm to the naive trie-building approach, we next sought to assess the scaling properties of the shortcut-enabled approach in isolation.

Figure 5 depicts the relationship between trie size as 100,000-tip batches are added and the time taken to insert each batch, over 5 replicates of 10 million tip reconstructions. Fitting a linear regression model to this data, we found significant evidence to reject the null hypothesis of a constant slope, for both the adaptive and purifying regime datasets ($\alpha = 0.05$). Evidence indicates, therefore, that as trie size increases, the time required per tip added increases as well.

Notably, batch processing times exhibit strong outliers for which trie extension was notably slower. This effect is due to the presence of expensive consolidation steps within those batches. To assess this phenomenon, we performed a more fine-grained analysis (with batches of 5,000 tips) on non-outlier data (filtering out values above the 95th percentile). Further regression analysis using this data suggests that, past a certain point, non-outlier batch times do not increase significantly with larger tree size. (See supplement for more information (Singhvi et al., 2025).) In an analytical investigation of scaling complexity, which is left to future work, we therefore expect the degree to which expensive trie consolidation operations are amortized over average-case insertions to play an important role in determining the scaling complexity of the shortcut-enabled approach.

Microbenchmarks measuring net reconstruction time across problem sizes, rather than marginal batch-wise throughput, reach a similar conclusion and are provided in Supplemental Figure S0.9 (Singhvi et al., 2025).

Macrobenchmark: Billion-Tip Reconstruction

Given the promising results of microbenchmark trials, particularly the favorable empirical scaling result, we next sought to assess the performance of our approach on an extremely large problem size.

For this purpose, we conducted trial reconstruction benchmarks comprising the full corpus of 1 billion agent genomes extracted from our trial Wafer-Scale Engine experiments. As a point of comparison, the largest reconstructions performed using the previous approach comprised only 10,000 tips (Moreno et al., 2024b).

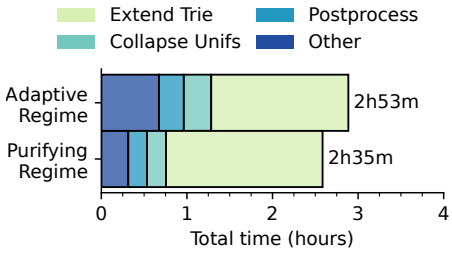


Figure 6: **Execution profile of reconstruction pipeline from macrobenchmark trial.** Results are shown for 1 billion tip reconstruction trials. Trials differed in simulation data source, with "adaptive regime" genomes sampled under conditions with lower phylogenetic richness. Other category includes file load and save time as well as hstrat marker explosion/decoding.

Benchmarked operations included the full reconstruction operation pipeline shown in Figure 1. Input data comprised a Parquet format file storing raw agent genomes as hexadecimal strings. The population file size was 34 GB for and 35 GB for the sample adaptive- and purifying-selection regimes, respectively. The output data comprised the reconstructed phylogeny in an edge-list format closely resembling the ALife data standard. Several pieces of metadata (e.g., agent position) were forwarded to the output data. For the adaptive-selection regime, the output file size was 54 GB, and for the purifying-selection regime, the output file size was 55 GB.

Given the large problem-size scale tested, macrobenchmark trials were carried out on a compute cluster node, rather than a desktop PC, as was used for microbenchmark experiments. Figure 6 profiles net runtime in our case study reconstruction trials. Reconstruction time was 2h:53m for the adaptive-regime data set and 2h:35m for the purifying-regime data set. These figures correspond to a net throughput of 5.8 and 6.4 million tips per minute, respectively. Peak memory use was 693 GB (adaptive regime) and 614 GB (purifying regime).

Validation: Comparison to Ground Truth

In a first set of trials to validate the quality of phylogenies produced by the shortcut-enabled algorithm, we tested inference accuracy against ground-truth data. For these trials, we conducted experiments using a neutral evolution model on a single CPU, where we were able to directly record the underlying phylogeny history. Example visualizations of corresponding reconstruction and ground-truth phylogenies

Downloaded from direct.mit.edu/isal/proceedings-pdf/isal2025/37/62/2567097/isal_a_890.pdf by guest on 14 October 2026

are provided in supplementary materials (Singhvi et al., 2025).

Surface Size	8	16	32	64
Mean Error	0.373	0.191	0.095	0.021
Standard Deviation	0.240	0.156	0.082	0.018

Table 1: **Accuracy of shortcut table algorithm on simulated data.** Mean error is defined as the average triplet distance between reconstructed and ground truth phylogenies over 20 random simulations of evolution, with each organism having a variable surface size. A steady retention policy was used.

Table 1 reports reconstruction error across annotation sizes ranging from 8 single-bit markers up to 64 single-bit markers. As expected, medium and large annotation sizes produced lowest reconstruction error, with large annotations of 64 bits achieving triplet distance error on average around 2%.[4] In contrast, very small 8-bit annotations averaged 37% reconstruction error.

Conclusion

We have presented and benchmarked a new algorithm for phylogenetic reconstruction from synthetic hstrat (Moreno & Dolson, 2024) data with significantly better performance, both empirically and asymptotically, rectifying the existing bottleneck of slow phylogenetic reconstruction in wafer-scale distributed simulations.

The scope of scientific questions tractable to investigation *in silico* continues to be broadened by rapid advances in computing technology. However, such advances also pose new challenges in managing vast quantities of data. By providing scalable means for fast phyloanalysis of large simulations, our work seeks to help the scientific utility of parallel and distributed digital evolution experiments keep pace with opportunities afforded by emerging hardware architectures.

In a parallel vein, the volume of data processed in bioinformatics workflows is also increasing with continuing advances in high-throughput sequencing technologies, enabling the construction of phylogenies containing millions of taxa. As an illustrative example at the cutting edge of extreme scale, Konno et al. (2022) reports phylogeny synthesis from 235 million sequence reads generated from an *in silico* CRISPR barcoding model — requiring 31 hours of compute time across 300 HPC nodes.

In both the context of bioinformatics and artificial life research, very large-scale phylogeny data enabled by advances in sequencing and computing technology represent a new challenge as much as an opportunity, raising the question of how best to mine this data. On a practical level, work is needed not just to push the boundaries of what can be learned from phylogenies, but also how to store, load, traverse, quantify, visualize, and manipulate very large phylogenies in an efficient manner. Indeed, projects are being developed to try to address this issue. For example, taxonium (Sanderson, 2022) is a web-based software for visualizing large phylogenies in a flexible, interactive manner, and is able to handle browsing millions of tips at a high frame rate. Other projects aim to create methods for compact, scalable phylogeny representations (Moshiri, 2025, 2020), enabling faster and more memory-efficient tree operations.

[4]This error measure corresponds to the fraction of sampled three-node sets for which both trees report the same two nodes as more closely related.

Future Work

In pushing the boundaries of phylogenetic scale to billion-tip datasets, ALife research has the opportunity to contribute to an interdisciplinary ecosystem of software tools developing around working with very large-scale phylogenies. In particular, the ALife data standard, which specifies a tabular representation for phylogeny data (Lalejini et al., 2019), has strong potential to contribute to a larger high-performance phylogeny processing infrastructure. Although originally envisioned as a data storage format, the tabular nature of the standard allows integrations with high-performance software tools built around the "dataframe" concept, including Pandas, Polars, Dask, and data.table. These libraries provide a structured, user-friendly interface to advanced performance features such as multithreading, data streaming, query optimization, file partitioning, and column-oriented binary file formats (McKinney, 2010; Barrett et al., 2025; Vink et al., 2024; Rocklin, 2015). Additionally, for Python users, the columnar array format typical in dataframe libraries is compatible with NumPy and Numba, readily enabling on-the-fly SIMD vectorization and just-in-time compilation (Harris et al., 2020; Lam et al., 2015). Indeed, this approach underlies much of the pre- and post-processing steps for end-to-end reconstruction demonstrated in this work.

Of more direct bearing to phylogeny reconstruction from heredity stratigraphy data, future work should also assess tradeoffs in configuring hstrat annotations for phylogeny reconstructions incorporating fossil data (including extinct lineages) drawn from earlier time points, as was the case in this work. Whereas existing analysis has focused on reconstructions from genomes drawn from a shared contemporary population (Moreno et al., 2025b), it is likely that configuration adjustments may be necessary for scenarios involving genomes of widely varying generational depths. Promisingly, in preliminary testing, we have found indications that incorporating fossil data into reconstructions can help improve inference accuracy of relationships among extant population members.

Other future work will involve experiments putting the high-throughput phylogeny reconstruction capabilities developed into practice. In ongoing work, we are interested in using large-scale digital evolution experiments to test phylostatistical methodologies for detecting signatures of multilevel selection associated with major transitions in evolution (Bonetti Franceschi & Volz, 2024), as well as pursuing hypothesis-driven experiments investigating mechanisms of open-ended evolution. To this end, all tools described herein are published as modular open source library components, supporting the broader research community in exploring and extending this line of inquiry.

Downloaded from direct.mit.edu/isal/proceedings-pdf/isal2025/37/62/2567097/isal_a_890.pdf by guest on 14 October 2026

Acknowledgment

Computational resources were provided by the MSU Institute for Cyber-Enabled Research and by the Pittsburgh Supercomputing Center Neocortex under NSF ACCESS Innovative Projects Allocation BIO240102 (Buitrago & Nystrom, 2021; Boerner et al., 2023). Any opinions, findings, and conclusions or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of the National Science Foundation.

This material is based upon work supported by the U.S. Department of Energy, Office of Science, Office of Advanced Scientific Computing Research (ASCR), under Award Number DE-SC0025634. This report was prepared as an account of work sponsored by an agency of the United States Government. Neither the United States Government nor any agency thereof, nor any of their employees, makes any warranty, express or implied, or assumes any legal liability or responsibility for the accuracy, completeness, or usefulness of any information, apparatus, product, or process disclosed, or represents that its use would not infringe privately owned rights. Reference herein to any specific commercial product, process, or service by trade name, trademark, manufacturer, or otherwise does not necessarily constitute or imply its endorsement, recommendation, or favoring by the United States Government or any agency thereof. The views and opinions of authors expressed herein do not necessarily state or reflect those of the United States Government or any agency thereof.

This material is based upon work supported by the Eric and Wendy Schmidt AI in Science Postdoctoral Fellowship, a Schmidt Sciences program.

References

Ackley, D. (2016). Indefinite scalability for living computation. *Proceedings of the AAAI Conference on Artificial Intelligence*, 30(1), 4142–4146. https://doi.org/10.1609/aaai.v30i1.9802

Ackley, D. & Small, T. (2014). Indefinitely scalable computing = artificial life engineering. *Artificial Life 14: Proceedings of the Fourteenth International Conference on the Synthesis and Simulation of Living Systems*, Alife, 606–613. https://doi.org/10.1162/978-0-262-32621-6-ch098

Ackley, D. H. (2023). A robust programmable replicator for an indefinitely scalable machine. *The 2023 Conference on Artificial Life*, Alife 2023. https://doi.org/10.1162/isal_a_00701

Barrett, T., Dowle, M., Srinivasan, A., Gorecki, J., Chirico, M., Hocking, T., Schwendinger, B., & Krylov, I. (2025). *data.table: Extension of 'data.frame'*. https://r-datatable.com. R package version 1.17.99, https://Rdatatable.gitlab.io/data.table, https://github.com/Rdatatable/data.table

Boerner, T. J., Deems, S., Furlani, T. R., Knuth, S. L., & Towns, J. (2023). Access: Advancing innovation: Nsf's advanced cyberinfrastructure coordination ecosystem: Services & support. *Practice and Experience in Advanced Research Computing*, Pearc '23, 173–176. https://doi.org/10.1145/3569951.3597559

Bonetti Franceschi, V. & Volz, E. (2024). Phylogenetic signatures reveal multilevel selection and fitness costs in sars-cov-2. *Wellcome Open Research*, 9, 85. https://doi.org/10.12688/wellcomeopenres.20704.2

Buitrago, P. A. & Nystrom, N. A. (2021). *Neocortex and Bridges-2: A High Performance AI+HPC Ecosystem for Science, Discovery, and Societal Good*, 205–219. Springer International Publishing. https://doi.org/10.1007/978-3-030-68035-0_15

Colijn, C. & Gardy, J. (2014). Phylogenetic tree shapes resolve disease transmission patterns. *Evolution, medicine, and public health*, 2014(1), 96–108.

Critchlow, D. E., Pearl, D. K., & Qian, C. (1996). The triples distance for rooted bifurcating phylogenetic trees. *Systematic Biology*, 45(3), 323–334.

Daudey, H., Parsons, D. P., Tannier, E., Daubin, V., Boussau, B., Liard, V., Gallé, R., Rouzaud-Cornabas, J., & Beslon, G. (2024). Aevol_4b: Bridging the gap between artificial life and bioinformatics. *The 2024 Conference on Artificial Life*, Alife 2024. https://doi.org/10.1162/isal_a_00716

De La Briandais, R. (1959). File searching using variable length keys. *Papers presented at the the March 3-5, 1959, western joint computer conference on XX - IRE-AIEE-ACM '59 (Western)*, IRE-AIEE-ACM '59 (Western), 295–298. https://doi.org/10.1145/1457838.1457895

Dolson, E. & Ofria, C. (2021). Digital evolution for ecology research: A review. *Frontiers in Ecology and Evolution*, 9. https://doi.org/10.3389/fevo.2021.750779

Dolson, E., Rodriguez-Papa, S., & Moreno, M. A. (2024). *Phylotrack: C++ and python libraries for in silico phylogenetic tracking*. https://doi.org/10.48550/arxiv.2405.09389

Emani, M., Raskar, S., John, J., Durillo, J. J., Ben-Nun, T., Van Essen, B., Ltaief, H., & Brown, N. (2024). *Democratizing ai accelerators for hpc applications: Challenges, success, and support*. https://sc24.conference-program.com/presentation/?id=bof133&=&sess=sess628

Foster, E. D. & Deardorff, A. (2017). Open science framework (osf). *Journal of the Medical Library Association*, 105(2). https://doi.org/10.5195/jmla.2017.88

Fredkin, E. (1960). Trie memory. *Communications of the ACM*, 3(9), 490–499. https://doi.org/10.1145/367390.367400

Downloaded from direct.mit.edu/isal/proceedings-pdf/isal2025/37/62/2567097/isal.a.890.pdf by guest on 14 October 2026

Genthon, A., Nozoe, T., Peliti, L., & Lacoste, D. (2023). Cell lineage statistics with incomplete population trees. *PRX Life*, 1(1). https://doi.org/10.1103/prxlife.1.013014

Haller, B. C. & Messer, P. W. (2023). Slim 4: Multispecies eco-evolutionary modeling. *The American Naturalist*, 201(5), E127–e139. https://doi.org/10.1086/723601

Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M. H., Brett, M., Haldane, A., del Río, J. F., Wiebe, M., Peterson, P., Gérard-Marchant, P., Sheppard, K., Reddy, T., Weckesser, W., Abbasi, H., Gohlke, C., & Oliphant, T. E. (2020). Array programming with numpy. *Nature*, 585(7825), 357–362. https://doi.org/10.1038/s41586-020-2649-2

Hernandez, J. G., Lalejini, A., & Dolson, E. (2022). Phylogenetic diversity predicts future success in evolutionary computation. *Proceedings of the Genetic and Evolutionary Computation Conference Companion*, Gecco '22, 23–24. https://doi.org/10.1145/3520304.3534079

Hunter, J. D. (2007). Matplotlib: A 2d graphics environment. *Computing in Science & Engineering*, 9(3), 90–95. https://doi.org/10.1109/mcse.2007.55

Konno, N., Kijima, Y., Watano, K., Ishiguro, S., Ono, K., Tanaka, M., Mori, H., Masuyama, N., Pratt, D., Ideker, T., Iwasaki, W., & Yachie, N. (2022). Deep distributed computing to reconstruct extremely large lineage trees. *Nature Biotechnology*, 40(4), 566–575. https://doi.org/10.1038/s41587-021-01111-2

Lalejini, A., Dolson, E., Bohm, C., Ferguson, A. J., Parsons, D. P., Rainford, P. F., Richmond, P., & Ofria, C. (2019). Data standards for artificial life software. *The 2019 Conference on Artificial Life*, Alife 2019. https://doi.org/10.1162/isal_a_00213

Lam, S. K., Pitrou, A., & Seibert, S. (2015). Numba: a llvm-based python jit compiler. *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC*, Sc15, 1–6. https://doi.org/10.1145/2833157.2833162

Levy, S. F., Blundell, J. R., Venkataram, S., Petrov, D. A., Fisher, D. S., & Sherlock, G. (2015). Quantitative evolutionary dynamics using high-resolution lineage tracking. *Nature*, 519(7542), 181–186. https://doi.org/10.1038/nature14279

Li, Z., Yang, W., Wu, P., Shan, Y., Zhang, X., Chen, F., Yang, J., & Yang, J.-R. (2024). Reconstructing cell lineage trees with genomic barcoding: approaches and applications. *Journal of Genetics and Genomics*, 51(1), 35–47. https://doi.org/10.1016/j.jgg.2023.05.011

Lie, S. (2023). Cerebras architecture deep dive: First look inside the hardware/software co-design for deep learning. *IEEE Micro*, 43(3), 18–30. https://doi.org/10.1109/mm.2023.3256384

McKinney, W. (2010). Data structures for statistical computing in python. *Proceedings of the 9th Python in Science Conference*, SciPy, 56–61. https://doi.org/10.25080/majora-92bf1922-00a

Miller, M. A., Pfeiffer, W., & Schwartz, T. (2010). Creating the cipres science gateway for inference of large phylogenetic trees. *2010 Gateway Computing Environments Workshop (GCE)*. https://doi.org/10.1109/gce.2010.5676129

Moreno, M. A. & Dolson, E. (2024). *mmore500/hstrat*. https://doi.org/10.5281/zenodo.10779245

Moreno, M. A., Dolson, E., & Ofria, C. (2022a). Hereditary stratigraphy: Genome annotations to enable phylogenetic inference over distributed populations. *The 2022 Conference on Artificial Life*, Alife 2022, 64. https://doi.org/10.1162/isal_a_00550

Moreno, M. A., Dolson, E., & Ofria, C. (2022b). hstrat: a python package for phylogenetic inference on distributed digital evolution populations. *Journal of Open Source Software*, 7(80), 4866. https://doi.org/10.21105/joss.04866

Moreno, M. A., Dolson, E., & Rodriguez-Papa, S. (2023). Toward phylogenetic inference of evolutionary dynamics at scale. *The 2023 Conference on Artificial Life*, Alife 2023, 568–668. https://doi.org/10.1162/isal_a_00694

Moreno, M. A., Dolson, E., Zaman, L., Singhvi, V., Yang, C., & Rodriguez Papa, S. (2025a). *hstrat*. https://doi.org/10.5281/zenodo.16898849

Moreno, M. A., Ranjan, A., Dolson, E., & Zaman, L. (2025b). Testing the inference accuracy of accelerator-friendly approximate phylogeny tracking. *2025 IEEE Symposium on Computational Intelligence in Artificial Life and Cooperative Intelligent Systems (ALIFE-CIS)*, 1–9. https://doi.org/10.1109/alife-cis64968.2025.10979833

Moreno, M. A., Rodriguez Papa, S., & Dolson, E. (2024a). *Analysis of phylogeny tracking algorithms for serial and multiprocess applications*. https://doi.org/10.48550/arXiv.2403.00246

Downloaded from direct.mit.edu/isal/proceedings-pdf/isal2025/37/62/2567097/isal.a.890.pdf by guest on 14 October 2026

Moreno, M. A., Singhvi, V., Wagner, J., Dolson, E., & Zaman, L. (2025c). *mmore500/hstrat-reconstruction-algo: v1.0.1*. https://doi.org/10.5281/zenodo.16898918

Moreno, M. A., Yang, C., Dolson, E., & Zaman, L. (2024b). Trackable agent-based evolution models at wafer scale. *The 2024 Conference on Artificial Life*, 87–98. https://doi.org/10.1162/isal_a_00830

Moreno, M. A., Yang, C., Dolson, E., & Zaman, L. (2025d). *async-ga: agent-based evolution on the cerebras wafer-scale engine (wse)*. https://doi.org/10.5281/zenodo.16898904

Moreno, M. A., Zaman, L., & Dolson, E. (2024c). *Structured downsampling for fast, memory-efficient curation of online data streams*. https://doi.org/10.48550/arXiv.2409.06199

Moshiri, N. (2020). Treeswift: A massively scalable python tree package. *SoftwareX*, 11, 100436. https://doi.org/10.1016/j.softx.2020.100436

Moshiri, N. (2025). Compacttree: a lightweight header-only c++ library and python wrapper for ultra-large phylogenetics. *Gigabyte*, 2025. https://doi.org/10.46471/gigabyte.152

Ofria, C. & Wilke, C. O. (2004). Avida: A software platform for research in computational evolutionary biology. *Artificial Life*, 10(2), 191–229. https://doi.org/10.1162/106454604773563612

pandas developers (2020). pandas-dev/pandas: Pandas. *Zenodo*. https://doi.org/10.5281/zenodo.3509134

Pennock, R. T. (2007). Models, simulations, instantiations, and evidence: the case of digital evolution. *Journal of Experimental & Theoretical Artificial Intelligence*, 19(1), 29–42.

Rocklin, M. (2015). Dask: Parallel computation with blocked algorithms and task scheduling. *Proceedings of the 14th Python in Science Conference*, SciPy, 126–132. https://doi.org/10.25080/majora-7b98e3ed-013

Sanderson, T. (2022). Taxonium, a web-based tool for exploring large phylogenetic trees. *eLife*, 11. https://doi.org/10.7554/elife.82392

Singhvi, V., Wagner, J., Dolson, E., Zaman, L., & Moreno, M. A. (2025). *A scalable trie building algorithm for high-throughput phyloanalysis of wafer-scale digital evolution experiments*. https://doi.org/10.17605/osf.io/63ucz

Stadler, T. (2013). Recovering speciation and extinction dynamics based on phylogenies. *Journal of Evolutionary Biology*, 26(6), 1203–1219. https://doi.org/10.1111/jeb.12139

Stroud, J. T. & Ratcliff, W. C. (2025). Long-term studies provide unique insights into evolution. *Nature*, 639(8055), 589–601. https://doi.org/10.1038/s41586-025-08597-9

Vink, R., de Gooijer, S., Beedie, A., Gorelli, M. E., nameex-haustion, Peters, O., Guo, W., Burghoorn, G., van Zundert, J., Hulselmans, G., Grinstead, C., Marshall, chielP, Turner-Trauring, I., Mitchell, L., Santamaria, M., Heres, D., Magarick, J., Genockey, K., ibENPC, deanm0000, Harbeck, H., Wilksch, M., eitsupi, Koutsouris, I., Leitao, J., van Gelderen, M., Brannigan, L., & Barbagiannis, P. (2024). *pola-rs/polars: Python polars 1.16.0*. https://doi.org/10.5281/zenodo.14244124

Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J., van der Walt, S. J., Brett, M., Wilson, J., Millman, K. J., Mayorov, N., Nelson, A. R. J., Jones, E., Kern, R., Larson, E., Carey, C. J., Polat, İ., Feng, Y., Moore, E. W., VanderPlas, J., Laxalde, D., Perktold, J., Cimrman, R., Henriksen, I., Quintero, E. A., Harris, C. R., Archibald, A. M., Ribeiro, A. H., Pedregosa, F., van Mulbregt, P., & SciPy 1.0 Contributors (2020). SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods*, 17, 261–272. https://doi.org/10.1038/s41592-019-0686-2

Wang, J.-T., Lin, Y.-Y., Chang, S.-Y., Yeh, S.-H., Hu, B.-H., Chen, P.-J., & Chang, S.-C. (2020). The role of phylogenetic analysis in clarifying the infection source of a covid-19 patient. *Journal of Infection*, 81(1), 147–178. https://doi.org/10.1016/j.jinf.2020.03.031

Waskom, M. L. (2021). seaborn: statistical data visualization. *Journal of Open Source Software*, 6(60), 3021. https://doi.org/10.21105/joss.03021

Wiser, M. J., Ribeck, N., & Lenski, R. E. (2013). Long-term dynamics of adaptation in asexual populations. *Science*, 342(6164), 1364–1367.

Yang, C., Wagner, J., Dolson, E., Zaman, L., & Moreno, M. A. (2025). *Downstream: efficient cross-platform algorithms for fixed-capacity stream downsampling*. https://doi.org/10.48550/arXiv.2506.12975

Downloaded from direct.mit.edu/isal/proceedings-pdf/isal2025/37/62/2567097/isal.a.890.pdf by guest on 14 October 2026